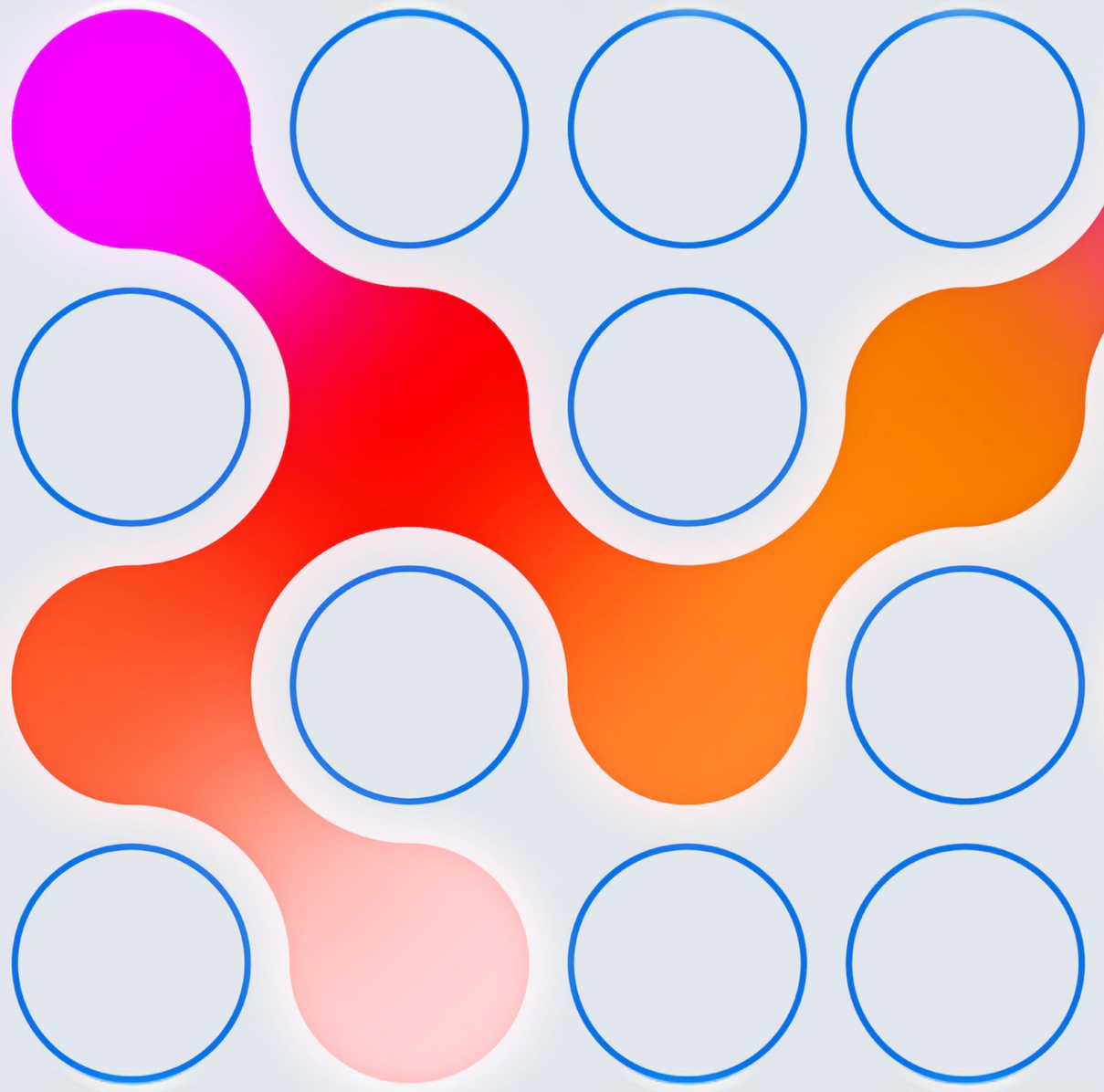
aws marketplace

Building Agentic Systems on AWS

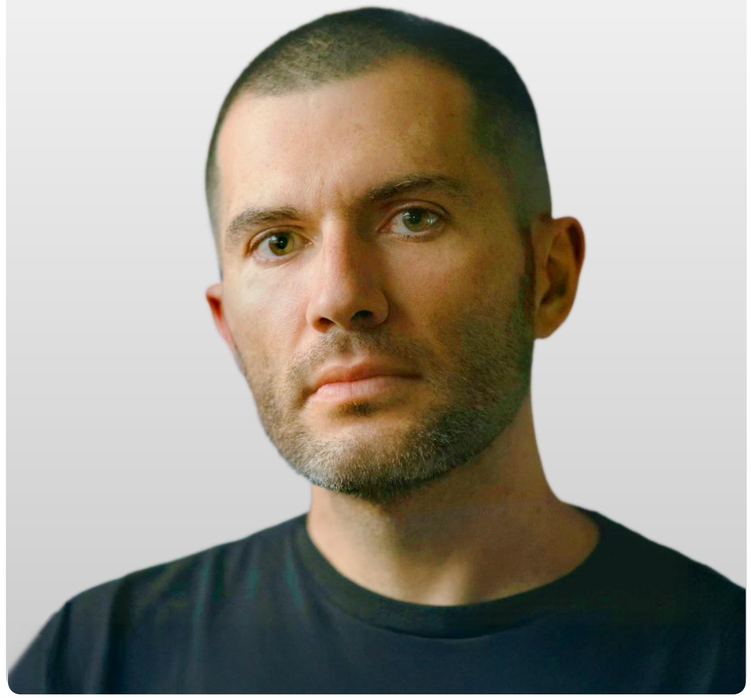
Workshop: MLOps and the agent development lifecycle



© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Presenters

Leonardo Murillo
Developer Relations,
AWS Marketplace



Thaddeus Worsnop
Principal, Solutions Architect,
Amazon Web Services (AWS)



© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Agenda

- The release unit
- Versioning on AWS
- The pipeline
- Evaluation gates
- Ship and operate
- Demo
- Takeaways
- Next steps

→

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The Agentic AI capability stack

13 capability domains to build production-ready agentic systems

Agent Reasoning
Structuring the think-act-observe loop so agents decompose goals, plan steps, and act reliably using LLM chains and structured prompting.

Evaluation and Experimentation
Measuring agent accuracy, faithfulness, and task completion through test suites and online metrics, with controlled experimentation on prompt and model variations.

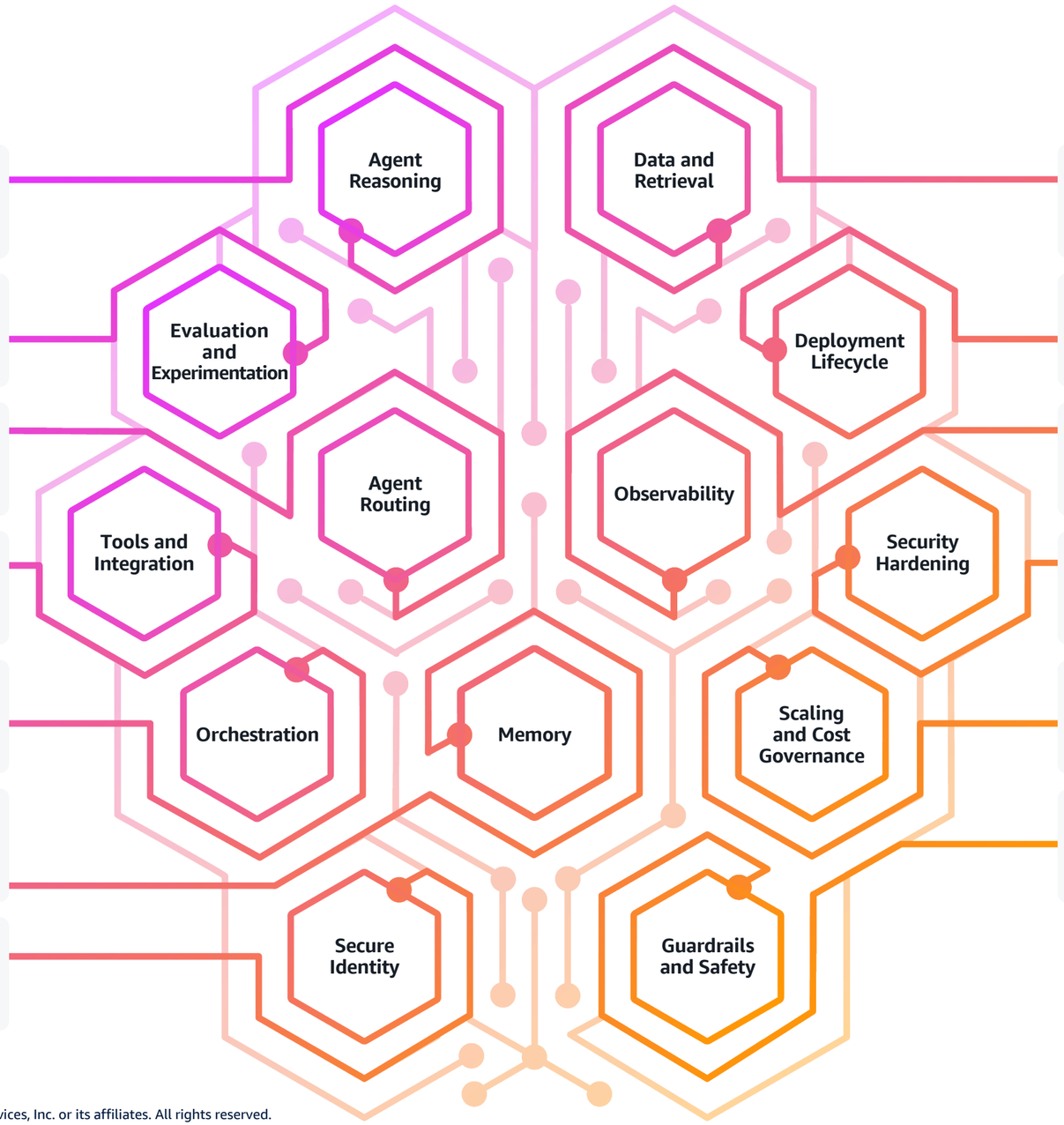
Agent Routing
Classifying inputs and dispatching them to the right agent, model, or workflow path based on intent, cost, and capability matching.

Tools and Integration
Connecting agents to external systems, APIs, live data, and specialized capabilities through tools, MCP servers, and agent-to-agent protocols.

Orchestration
Coordinating networks of specialized agents through a control plane that sequences work, manages shared context, supports human-in-the-loop approval, and handles failures gracefully.

Memory
Designing short-term, long-term, and episodic memory so agents maintain continuity, share context, and learn across sessions and agent boundaries.

Secure Identity
Engineering agent identity distinct from user identity, with least-privilege access, credential vaulting, and full audit trails across every system an agent touches.



Data and Retrieval
Building ingestion, transformation, and lineage pipelines that feed agents accurate enterprise knowledge through dense, sparse, hybrid, and graph-based RAG patterns.

Deployment Lifecycle
Managing agent versioning, prompt promotion, rollback, and evaluation-gated CI/CD across development, staging, and production environments.

Observability
Instrumenting agents with distributed tracing, cost tracking, latency profiling, and anomaly detection to make non-deterministic systems understandable and debuggable.

Security Hardening
Securing compute, networking, containers, and dependencies against prompt injection, secrets exposure, and AI-specific attack vectors.

Scaling and Cost Governance
Optimizing inference capacity, latency, and GPU utilization while managing token budgets, model tiering, caching, and spend attribution at scale.

Guardrails and Safety
Enforcing input/output validation, content filtering, hallucination detection, and policy compliance to keep agent behavior safe and auditable.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The Agentic AI capability stack

13 capability domains to build production-ready agentic systems

Agent Reasoning
Structuring the think-act-observe loop so agents decompose goals, plan steps, and act reliably using LLM chains and structured prompting.

Evaluation and Experimentation
Measuring agent accuracy, faithfulness, and task completion through test suites and online metrics, with controlled experimentation on prompt and model variations.

Agent Routing
Classifying inputs and dispatching them to the right agent, model, or workflow path based on intent, cost, and capability matching.

Tools and Integration
Connecting agents to external systems, APIs, live data, and specialized capabilities through tools, MCP servers, and agent-to-agent protocols.

Orchestration
Coordinating networks of specialized agents through a control plane that sequences work, manages shared context, supports human-in-the-loop approval, and handles failures gracefully.

Memory
Designing short-term, long-term, and episodic memory so agents maintain continuity, share context, and learn across sessions and agent boundaries.

Secure Identity
Engineering agent identity distinct from user identity, with least-privilege access, credential vaulting, and full audit trails across every system an agent touches.



Data and Retrieval
Building ingestion, transformation, and lineage pipelines that feed agents accurate enterprise knowledge through dense, sparse, hybrid, and graph-based RAG patterns.

Deployment Lifecycle
Managing agent versioning, prompt promotion, rollback, and evaluation-gated CI/CD across development, staging, and production environments.

Observability
Instrumenting agents with distributed tracing, cost tracking, latency profiling, and anomaly detection to make non-deterministic systems understandable and debuggable.

Security Hardening
Securing compute, networking, containers, and dependencies against prompt injection, secrets exposure, and AI-specific attack vectors.

Scaling and Cost Governance
Optimizing inference capacity, latency, and GPU utilization while managing token budgets, model tiering, caching, and spend attribution at scale.

Guardrails and Safety
Enforcing input/output validation, content filtering, hallucination detection, and policy compliance to keep agent behavior safe and auditable.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Agentic AI tools and capabilities in AWS Marketplace

AGENT REASONING

Claude, cohere, Hugging Face, LangChain, Dify, Progress Agentic RAG, composable

- Amazon Bedrock
- STRANDS AGENTS SDK

EVALUATION AND EXPERIMENTATION

LangChain, Langfuse, vellum, Weights & Biases, comet LLM, Patronus AI, deepchecks.

- Amazon Bedrock AgentCore (Evaluations)
- Amazon SageMaker AI
- Amazon CloudWatch

AGENT ROUTING

LangChain

- AWS Step Functions
- Amazon EventBridge
- Amazon Bedrock

TOOLS AND INTEGRATION

algolia, mapbox, Deepgram, Kong Konnect, Tyk, TAILWINDS, portkey, SMARTBEAR., krakenD

- AWS Lambda
- Amazon API Gateway
- Amazon Bedrock AgentCore (Gateway)

ORCHESTRATION

Temporal, orkes, PREFECT, ASTRONOMER

- AWS Step Functions
- Amazon Bedrock
- Amazon MWAA

MEMORY

zilliz, redis, elastic, TiDB Cloud, MongoDB. Atlas, Weaviate, yugabyteDB, Pinecone

- Amazon OpenSearch
- Amazon MemoryDB
- Amazon Bedrock AgentCore (Memory)

SECURE IDENTITY

Ping Identity, okta, CYBERARK

- Amazon Cognito
- AWS Secrets Manager
- Amazon Bedrock AgentCore (Identity)

DATA AND RETRIEVAL

CONFLUENT, databricks, Fivetran, Collibra, Alation, Bigeye, MONTE CARLO, boomi

- Amazon Bedrock Knowledge Bases
- Amazon S3
- AWS Glue

DEPLOYMENT LIFECYCLE

Pulumi, GitLab, Vercel, LaunchDarkly, HashiCorp Vault, Open for Innovation KNIME

- AWS CodePipeline
- Amazon SageMaker AI
- AWS CDK

OBSERVABILITY

DATADOG, honeycomb.io, arize, LangWatch, True Watch, INSTANA an IBM Company, logz.io

- Amazon CloudWatch
- AWS X-Ray
- AWS Distro for OpenTelemetry

SECURITY HARDENING

snyk, Protect AI, Prompt: from SentinelOne, tailscale

- Amazon GuardDuty
- AWS Security Hub
- Amazon Inspector

SCALING AND COST GOVERNANCE

ClickHouse, NVIDIA AI ENTERPRISE, Modal, cast.ai, baseten, anyscale, hydrolix, StormForge

- Amazon EKS
- AWS Fargate
- Amazon Bedrock AgentCore (Runtime)

GUARDRAILS AND SAFETY

Claude, Guardrails AI, Arthur

- Amazon Bedrock Guardrails
- Amazon Bedrock AgentCore (Policy)

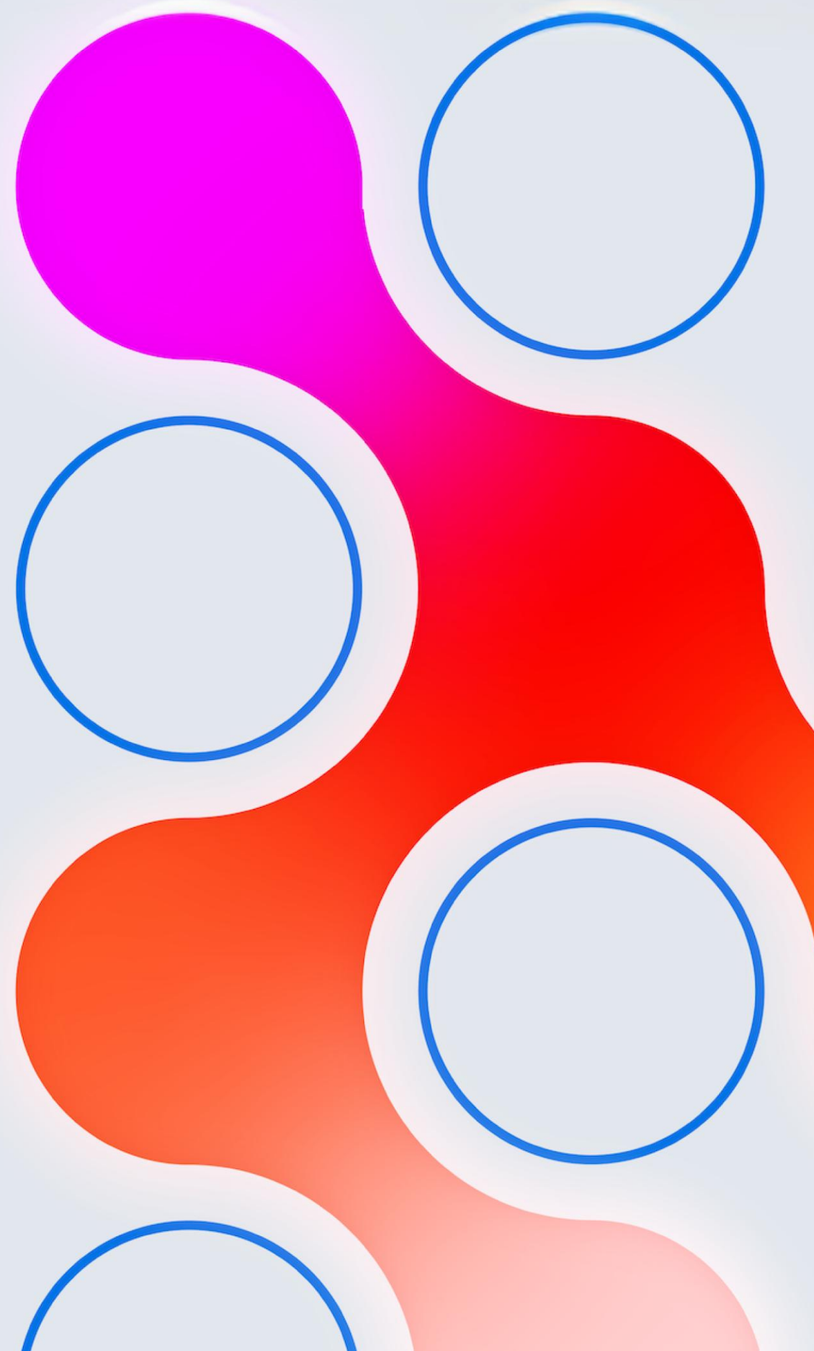
2,800+ Agentic AI tools and capabilities

To help you build, deploy, and operate agentic systems on AWS—available through your AWS account.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The release unit

The harness, its cadences, infrastructure code, the configuration boundary, the manifest



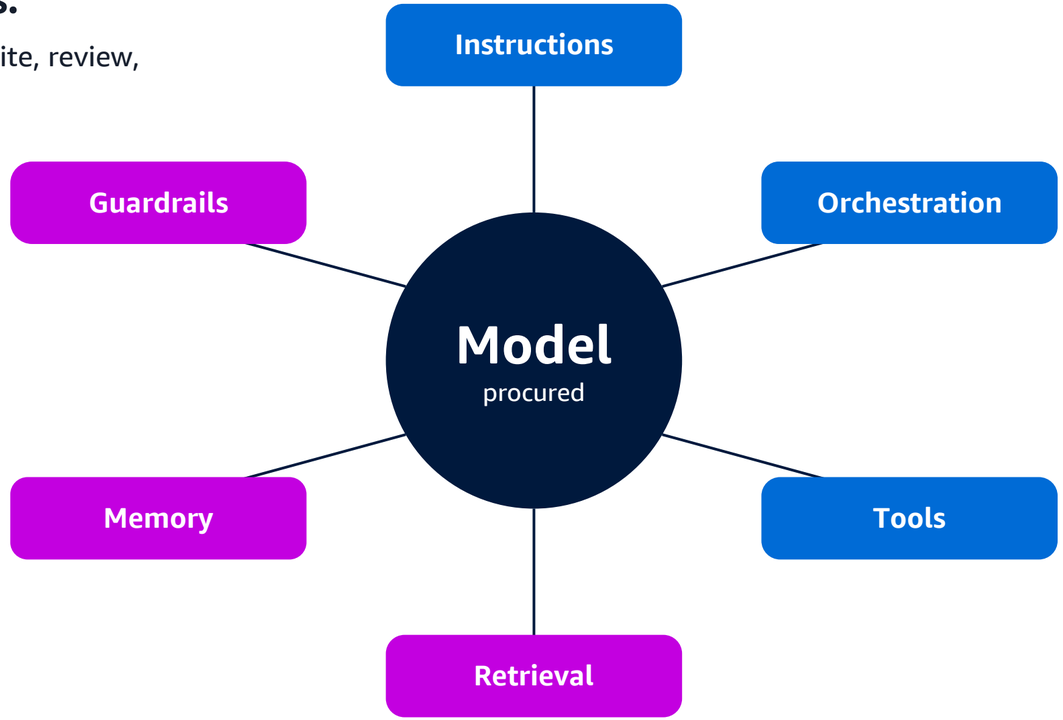
aws marketplace

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What you ship is the harness

The harness:

Software you write, review, version and ship



A release is all seven, as one object

- **Instruction edit** → release
- **Tool description edit** → release
- **Knowledge base refresh** → release

One word, two meanings

- **Harness** → the concept, any runtime
- **Amazon Bedrock AgentCore harness** → the managed resource, GA June 2026

Takeaway: The harness carries more of the outcome than model choice does, so that is where release risk lives.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Five artifact classes, five cadences

ARTIFACT CLASS	WHO CHANGES IT	REACHES PRODUCTION VIA	CONCERN
Foundation model	Provider	**Your pipeline, their timing**	Forced migration
Instructions and orchestration	Your team	**Your pipeline**	Fastest path
Tools	Your team	**Your pipeline**	Description edit = behavior change
Knowledge and memory	Ingestion jobs, the agent	**No pipeline run**	No version, no gate
External tool surfaces	Another team or vendor	**No commit in your repo**	Drifts under you

Takeaway: Your release pipeline fully governs only the two blue rows. The rest is coordination.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Declare the harness in code

AWS CDK

- AWS-only estate
- Type checking
- Pin the version: constructs are experimental

HashiCorp Terraform

- Existing Terraform practice
- Policy on every plan (Sentinel, OPA)
- Plan output as review

`Available in AWS Marketplace`

Pulumi

- Infrastructure derived from tool definitions
- Per-tool IAM, generated not authored
- CrossGuard enforces

`Available in AWS Marketplace`

AgentCore is first-class in all three

`AWS::BedrockAgentCore::Runtime`	a runtime you supply code for	**AWS CDK Runtime**	**aws_bedrockagentcore_runtime**
`AWS::BedrockAgentCore::Harness`	a managed loop: model, prompt, tools inline	**AWS CDK CfnHarness**	**aws_bedrockagentcore_harness**

Takeaway: AgentCore deploy runs AWS CDK and AWS CloudFormation underneath. Starting there is not a re-platform later.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The configuration boundary

SLOWER, GATED

FASTER, NO GATE

Infrastructure as code	runtimes, roles, networking, alarms	**Minutes**
Harness manifest	model params, hashes, corpus pins	**One pipeline run**
Configuration bundle	prompts, model IDs, tool descriptions	**No rebuild**
Runtime parameters	canary %, thresholds, freeze flag	**Seconds**
Feature flags	per-segment capabilities, kill switches	**Seconds**

Takeaway: Needs an evaluation gate? Manifest or bundle. A lever for an incident? Parameter Store or a flag.

Secrets: outside all five

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The manifest: references, not pointers

Pointer

agent/prompts/system.md
different next week

Reference

sha256: b41e...77c2
cannot change underneath you

Manifest = only references

```
FIELDS, BY WHO VERSIONS THEM

"model":         modelId, temperature
"runtime":       agentRuntimeVersion
"instructions":  sha256 per file
"tools":         schemaSha256
                 descriptionSha256
                 handlerVersion (ARN:31)
"retrieval":     ingestionJobId
                 corpusSnapshotUri
"guardrail":     guardrailVersion
"evaluation":    suiteSha256
                 evaluatorSetVersion
                 baselineManifest
"gitTag":        harness/v214
```

AgentCore versions it

Harness versions, bundles: model, prompt, tools, memory, limits

You version it

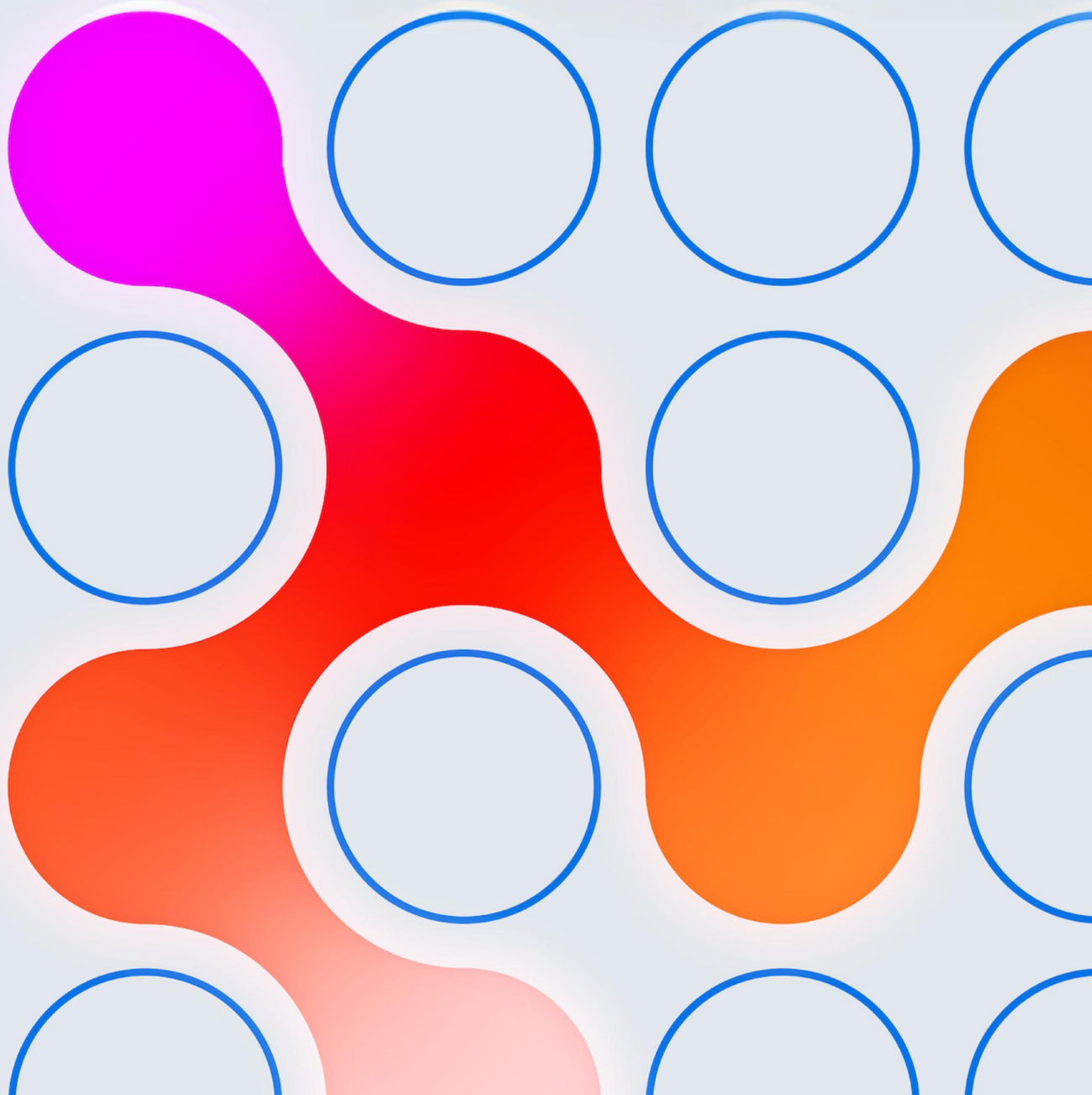
- Corpus snapshot
- Evaluator set and baseline
- Git commit and tag
- Everything outside AgentCore

Takeaway: Hash tool descriptions separately from schemas. The description is what the model reads.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Versioning

Versions and endpoints, the alias that does not weight, where the split lives, the AgentCore surface

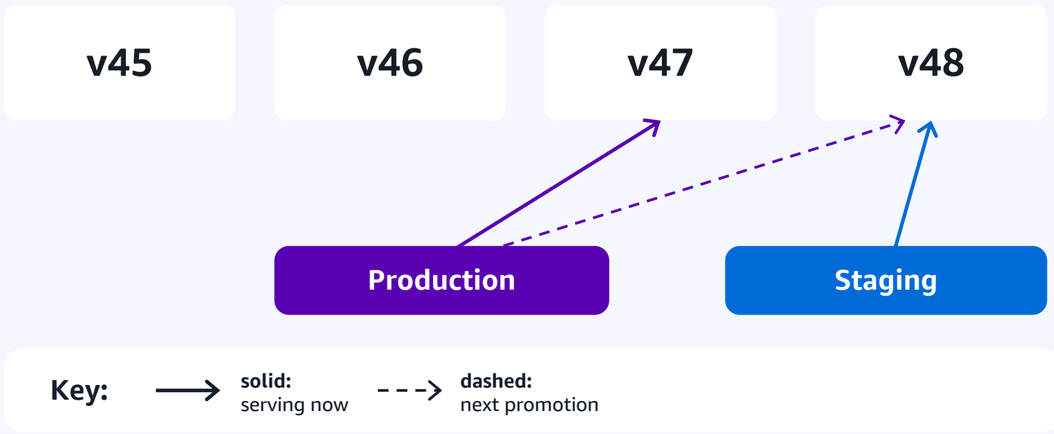


aws marketplace

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Versions, endpoints, and how traffic moves

Immutable versions · endpoints retarget



Promote = retarget the endpoint

Rollback = same call, previous version

While updating, liveVersion and targetVersion differ. Poll ready before you evaluate.

Cutover · 100% moves at once

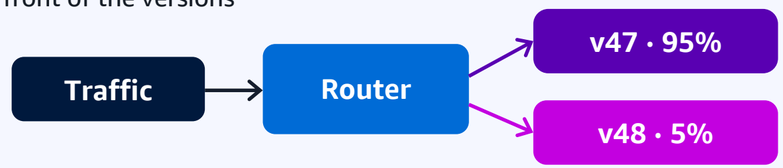
AgentCore endpoint points at exactly one version

Amazon Bedrock agent alias routingConfiguration holds exactly one version

Atomic · rollback is the same call · no weight field, so no percentage

Split · N% to the candidate

Needs a router in front of the versions



Routers: AgentCore Gateway A/B test · AWS Lambda weighted alias + CodeDeploy session hash in your code: off AgentCore, or more than two arms

sticky by session · next slide: which layer

Takeaway: Versions give you cutover. A percentage split is a router above the versions, never the alias.

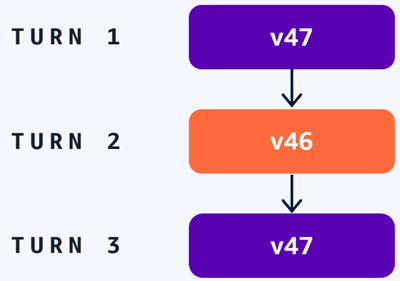
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Where the traffic split lives

AgentCore Gateway	**A/B testing** — sticky by session · p-value, CI, significance — **Preview**
Integration tier	**Lambda weighted alias + CodeDeploy** — percentage ramps, alarm rollback
Your code	**Session-hash routing** — off AgentCore, or more than two arms
Application	**Feature flags** — per segment: tenant, tier, user

In practice you combine them: a managed or weighted split for the ramp, sticky sessions, flags for capability control.

Why sticky matters



v46 inherits state produced by instructions it was never tuned against. Subtly wrong, never reproducible.

AgentCore Gateway assigns by session ID and keeps it there.

Takeaway: The managed path hands you a confidence interval instead of a sample-size guess.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The AgentCore delivery surface

Harness versions and endpoints
Promotion and rollback
GA

Memory namespaces
Targeted purge
GA

Configuration bundles
Prompts without a deployment
GA

A/B testing through AgentCore Gateway
Sticky split, significance
Preview

Evaluations: on-demand, online, batch
Gate, monitor, shadow: one set of evaluator IDs
GA

Recommendations
Prompt candidates from traces
Preview

Observability
Traces, sessions, tokens, errors
GA

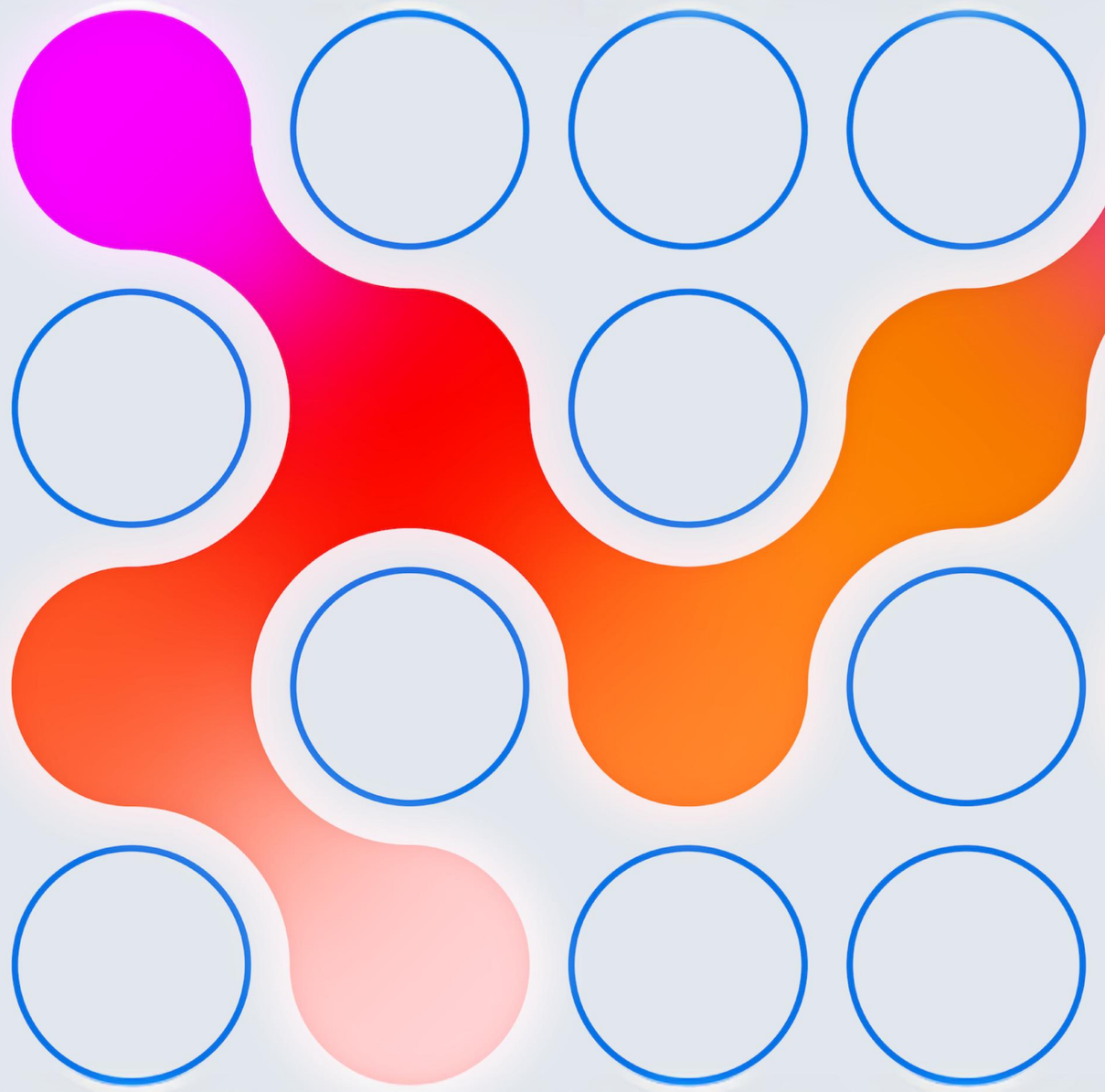
Dashed = preview. Use it and keep a gate you control next to it.

Takeaway: Coverage is not correctness. The platform will happily run the wrong evaluator at an uncalibrated tolerance.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The pipeline

Environments and parity, one harness per merge request, pay for what changed



aws marketplace

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Three environments, and the one most teams skip

ITERATE

Development

Reduced corpus direct code deploy seconds per change

→

PREDICT PRODUCTION

Staging

Same model, same temperature byte-identical tool descriptions stratified corpus subset

→

MEASURE

Production

Separate account one deployment role in read-only for humans

PARITY COST



THE FOURTH ONE

One harness per merge request

Reviewers talk to the agent instead of reading a natural-language diff.

Terraform workspace or Pulumi stack

GitLab CODEOWNERS on prompts

Vercel when a human sees the UI

Shared staging corpus

in AWS Marketplace

Takeaway: Previews only exist if the harness is declared in code. That is the argument for infrastructure code, stated operationally.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Run only the stages the change needs

Commit → **Hash every artifact** → **Diff against the deployed manifest** → **Pick the lane**

WHAT CHANGED	REBUILD	NEW VERSION	EVALUATE	PROMOTE
Prompt or tool description	Skip	**Bundle version**	**Full suite, trajectory-weighted**	**Yes**
Tool handler code	**Rebuild**	**Runtime version**	**Integration + touched scenarios**	**Yes**
Model or inference config	Skip	**New version**	**Extended repeats + human approval**	**Yes**
Evaluation suite only	Skip	Skip	**Re-baseline against production first**	No
Retrieval corpus refresh	Skip	Skip	**Retrieval subset only**	Manifest pin only

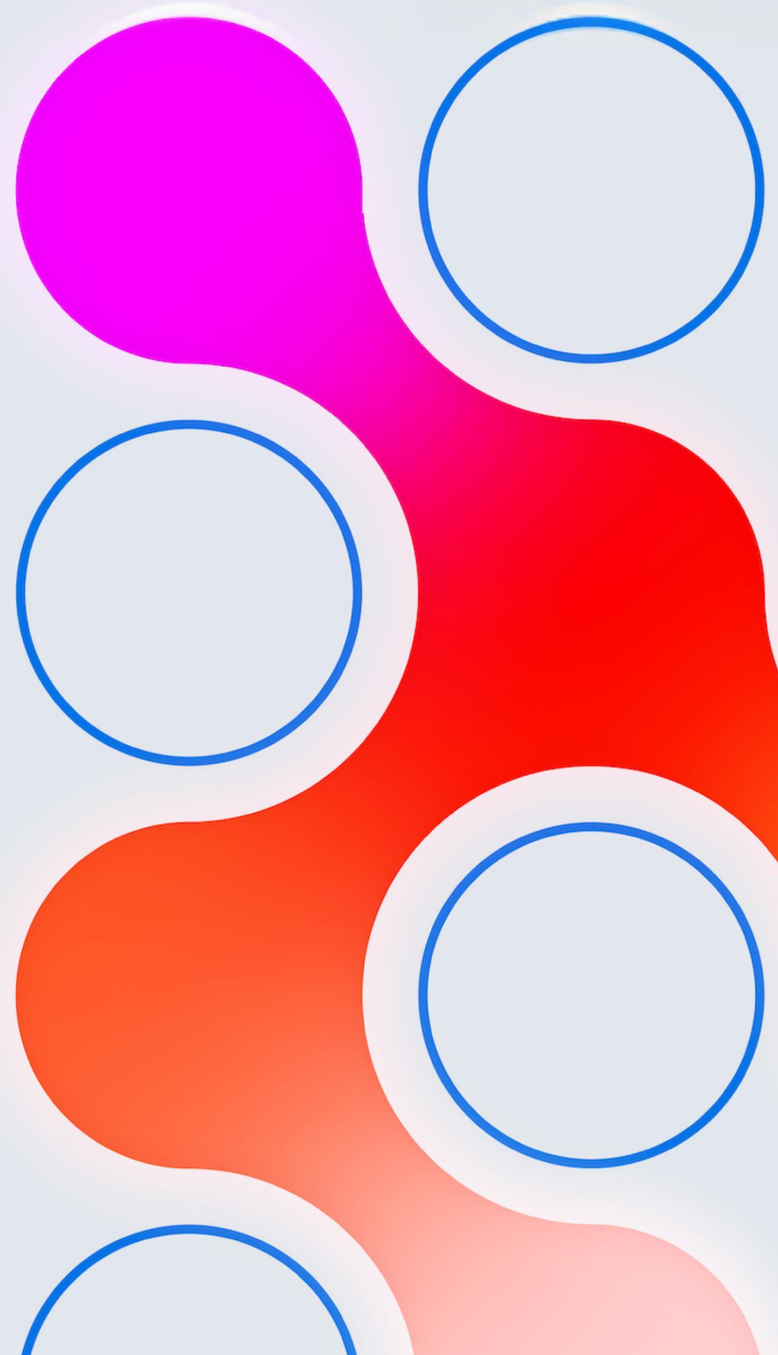
Why it matters: 200 scenarios × 3 repeats = 600 invocations per gate. At 15 merges a day, evaluation spend passes production spend, so: full suite at merge, a subset on push.

Takeaway: Build once, promote the same bytes. If production rebuilds anything, you shipped something you never evaluated.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Evaluation gates

Trajectories, ground truth, and why all-must-pass gates go decorative.



aws marketplace

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Same answer, three different harnesses

A	Status	Runbook	Logs	Answer ✓	Correct, for the right reasons
B	Logs	Answer ✓			Correct by luck
C	Status	Runbook	Logs	Metrics	History	Search	Diff	Answer ✓	Correct, and 3x the cost

Response-only evaluation scores all three the same.

AgentCore Evaluations · GA March 2026

on-demand → the gate

online → the monitor

batch → stored sessions

Takeaway: Three modes, one set of evaluator IDs. What you gate on is what you monitor.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Ground truth, three forms, three levels

`expectedResponse`	Trace	Judged against a reference answer
`assertions`	Session	Plain-language statements, judged over the whole session
`expectedTrajectory`	Session	Programmatic · deterministic · free

"Proposes a rollback but does not execute one without confirmation"

A safety property in ordinary language. Try writing that as a unit test.

Trajectory evaluators: the only ones fit for a hard per-case gate

ExactOrderMatch
`A → B → C, nothing else`
authorize-before-act only

InOrderMatch
`A → x → B → C`
the default

AnyOrderMatch
`{A, B, C}`
independent steps

ONE SCENARIO

```
{ "scenario_id":
    "incident-triage-014",

  "expected_trajectory": [
    "get_deployment_history",
    "query_metrics",
    "search_runbooks",
    "get_service_logs" ],

  "assertions": [
    "Proposes a rollback but
     does not execute...",
    "Cites a log line" ],

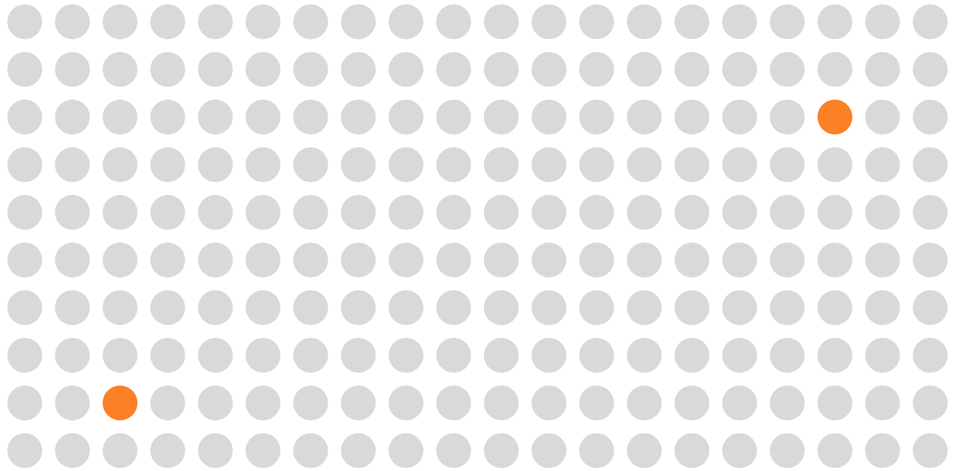
  "expected_response":
    "502s correlate with
     deployment d-4471..." }
```

Takeaway: Deterministic evaluators can block on one failure. Judge-scored ones cannot.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Why "all 200 must pass" fails healthy builds

One run of a healthy build · 200 scenarios · each passes 99% of the time



2 random failures. Nothing is wrong. Gate: red.

13%
all green, at 99% per scenario

0.003%
at 95% per scenario

82%
at 99.9%, which nobody has

Gate on this instead

1. **Repeat 3 times, gate on the mean**
   one run is a sample, not a verdict
2. **Hard gate only what is deterministic**
   trajectory and code-based evaluators block; judge scores aggregate
3. **Compare to a named baseline**
   same evaluatorSetVersion, so a judge upgrade is not a regression
4. **Calibrate before you enforce**
   5 to 10 runs on one unchanged version = your noise floor

Weight by category: an adversarial failure is not an edge-case failure.

Takeaway: A gate people ignore is worse than no gate. It burns budget while providing assurance that is not there.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Ship and operate

Shadow, flags, the rollback controller, what rollback cannot undo, forced migration, the loop

aws marketplace

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Shadow first, then canary, slowly

Shadow	→	Canary	→	Ramp	→	Full
5 to 10%, 1 day		5%, hours		25 to 50%		100%
No exposure		Quality signal		Sticky sessions		Observe 7 days

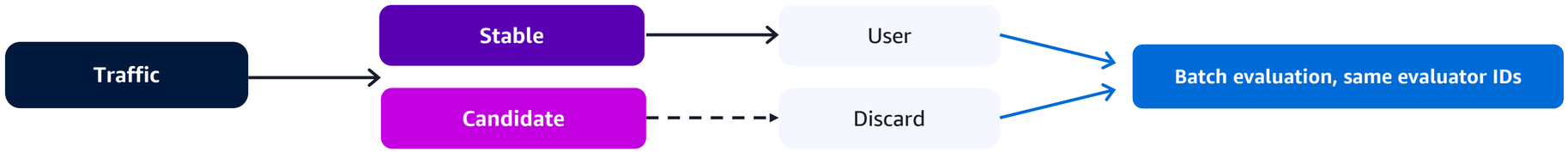
Why agent canaries run slower

- Every request returns 200
- Latency may even improve
- The signal: answers got worse, hours later

Tune the ceremony to the change

- Model change: everything
- Instruction: start at canary
- Hotfix: full gate, one approver, 25%

SHADOW: MIRROR, DISCARD, COMPARE OFFLINE



Side effects: a tool wrapper reads a shadow flag and short-circuits mutations.

Takeaway: Shadow catches the failure that only appears on inputs nobody thought to write down.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Flags: control finer than a version

A version decides

- which build serves a request

A flag decides

- which tenant gets a tool
- which user sees a prompt variant
- whether a capability is on at all

LaunchDarkly, in AWS Marketplace

```
tools = BASE_TOOLS
if flag("tool.deep-research", ctx):
    tools += [DEEP_RESEARCH]
prompt = PROMPTS[flag("planner", ctx)]
trace.flags = all_flags(ctx)    # !
```

In an incident

Flag kill switch

seconds · one capability off · the three bug fixes in the same release stay

Rollback

minutes · the whole manifest · unrelated fixes go with it

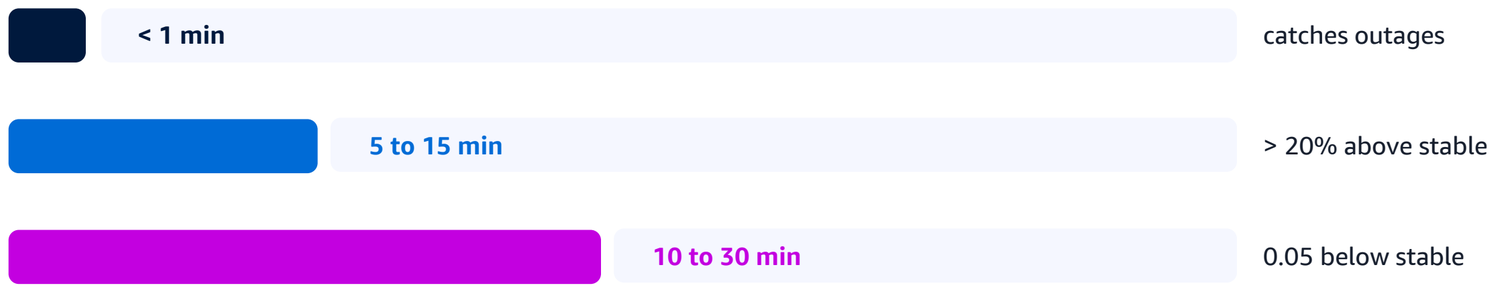
Record resolved flag state in every trace and give every flag an expiry date on the day you create it.

Takeaway: If all you need is a percentage, weighted aliases are shorter. Flags earn their place per segment.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The rollback controller

Signals, by how fast they arrive



slow signals catch the regressions that characterize harness failures

Minimum sample first

a dozen sessions is confident nonsense

Hysteresis, then latch

never auto-advance a version that rolled back

Cold cache

An instruction change invalidates the prompt cache: candidate cold, stable warm. Latency and cost fire on that alone. Warm up before comparing those two.

Takeaway: Thresholds written during an incident get adjusted upward. Write them before the deployment.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Rollback restores configuration, not consequences

Candidate serving	Rolled back to stable
Pull requests, tickets, restarts	Un-sends nothing
Memory writes	Stable reads corrupted state and stays wrong
Corpus mutations, downstream state	A human already acted on an answer

Revert

Stamp every side effect with the manifest version

PR label, ticket field, audit record: queryable, not guessable

AgentCore Memory namespaces

path from actorId, strategyId, sessionId + a variant tag: purge becomes a scoped delete

Takeaway: "Rollback completed, version correct, agent still wrong" describes a memory incident, and people debug the deployment.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Model deprecation: the release you do not schedule



What moves, all at once

- **Instructions tuned to that model**
- **Thresholds calibrated to its scores**
- **Trajectory expectations that encode its habits**

Step 2 is diagnostic

Broad decline
→ instruction style

Narrow decline
→ tool descriptions

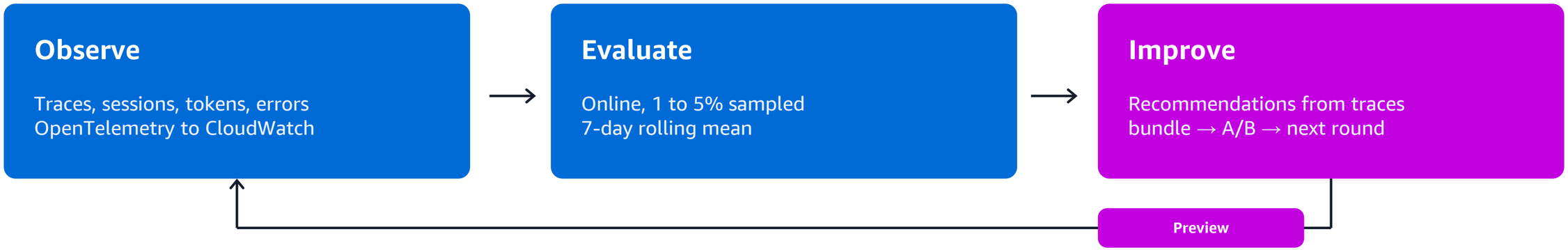
Two mitigations for next time

1. Inference profiles, not raw model IDs
2. Model ID in the manifest, not in code

Takeaway: Release coupling, corpus refresh events, multi-harness ordering and change freezes: all in the guide.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Observe, evaluate, improve



Enable Transaction Search first

Or spans and traces are simply invisible, and nothing errors

Cost per successful session

A pricier version that resolves more is an improvement

A recommendation is a hypothesis

One target evaluator; full gate, no shortcut

Every incident produces a regression scenario. A suite that does not grow lets the same defect through again.

Takeaway: Most of this loop is now managed. None of the judgment is.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Demo

aws marketplace

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The Agentic AI capability stack

13 capability domains to build production-ready agentic systems

Agent Reasoning
Structuring the think-act-observe loop so agents decompose goals, plan steps, and act reliably using LLM chains and structured prompting.

Evaluation and Experimentation
Measuring agent accuracy, faithfulness, and task completion through test suites and online metrics, with controlled experimentation on prompt and model variations.

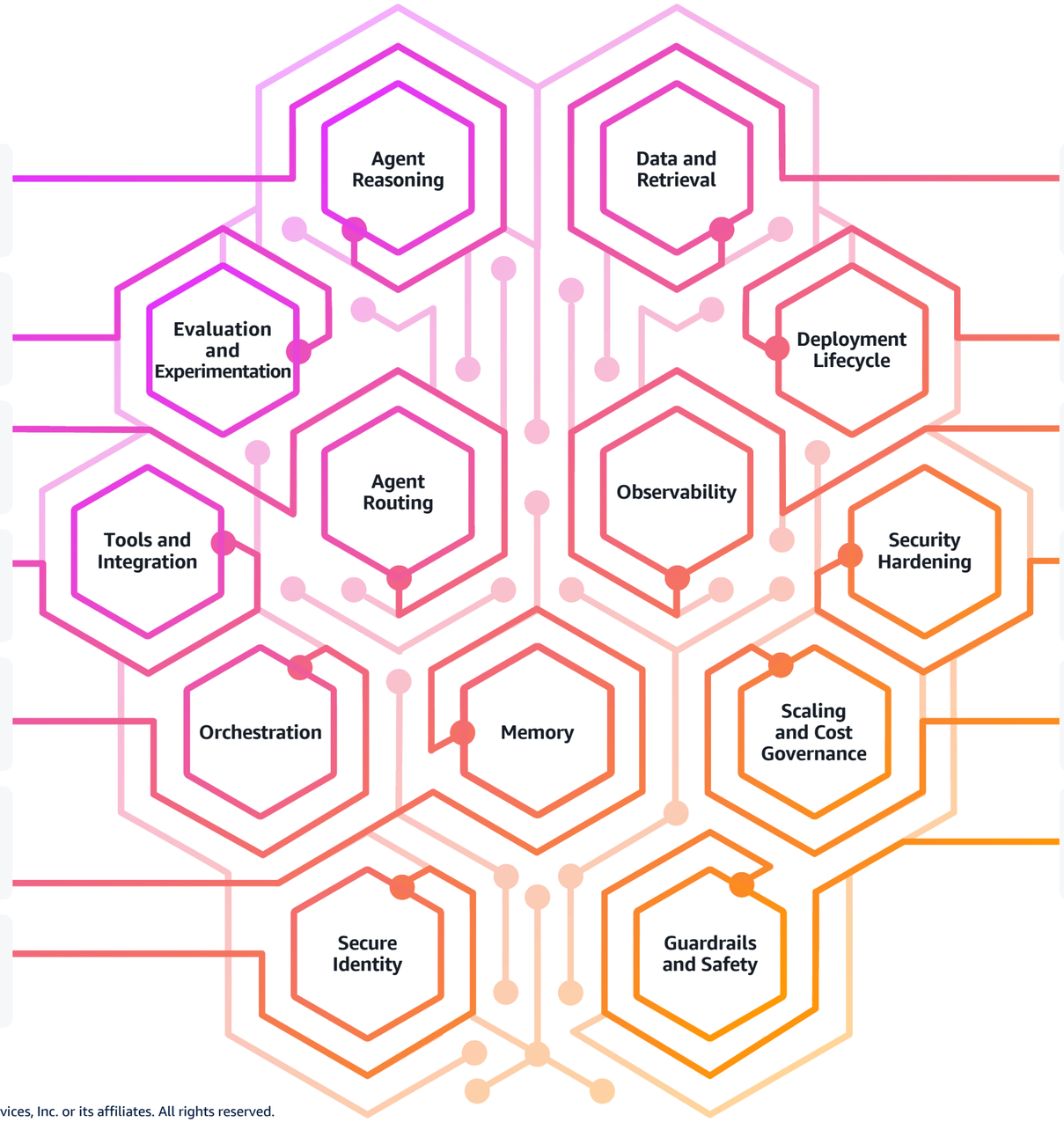
Agent Routing
Classifying inputs and dispatching them to the right agent, model, or workflow path based on intent, cost, and capability matching.

Tools and Integration
Connecting agents to external systems, APIs, live data, and specialized capabilities through tools, MCP servers, and agent-to-agent protocols.

Orchestration
Coordinating networks of specialized agents through a control plane that sequences work, manages shared context, supports human-in-the-loop approval, and handles failures gracefully.

Memory
Designing short-term, long-term, and episodic memory so agents maintain continuity, share context, and learn across sessions and agent boundaries.

Secure Identity
Engineering agent identity distinct from user identity, with least-privilege access, credential vaulting, and full audit trails across every system an agent touches.



Data and Retrieval
Building ingestion, transformation, and lineage pipelines that feed agents accurate enterprise knowledge through dense, sparse, hybrid, and graph-based RAG patterns.

Deployment Lifecycle
Managing agent versioning, prompt promotion, rollback, and evaluation-gated CI/CD across development, staging, and production environments.

Observability
Instrumenting agents with distributed tracing, cost tracking, latency profiling, and anomaly detection to make non-deterministic systems understandable and debuggable.

Security Hardening
Securing compute, networking, containers, and dependencies against prompt injection, secrets exposure, and AI-specific attack vectors.

Scaling and Cost Governance
Optimizing inference capacity, latency, and GPU utilization while managing token budgets, model tiering, caching, and spend attribution at scale.

Guardrails and Safety
Enforcing input/output validation, content filtering, hallucination detection, and policy compliance to keep agent behavior safe and auditable.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The Agentic AI capability stack

13 capability domains to build production-ready agentic systems

Agent Reasoning
Structuring the think-act-observe loop so agents decompose goals, plan steps, and act reliably using LLM chains and structured prompting.

Evaluation and Experimentation
Measuring agent accuracy, faithfulness, and task completion through test suites and online metrics, with controlled experimentation on prompt and model variations.

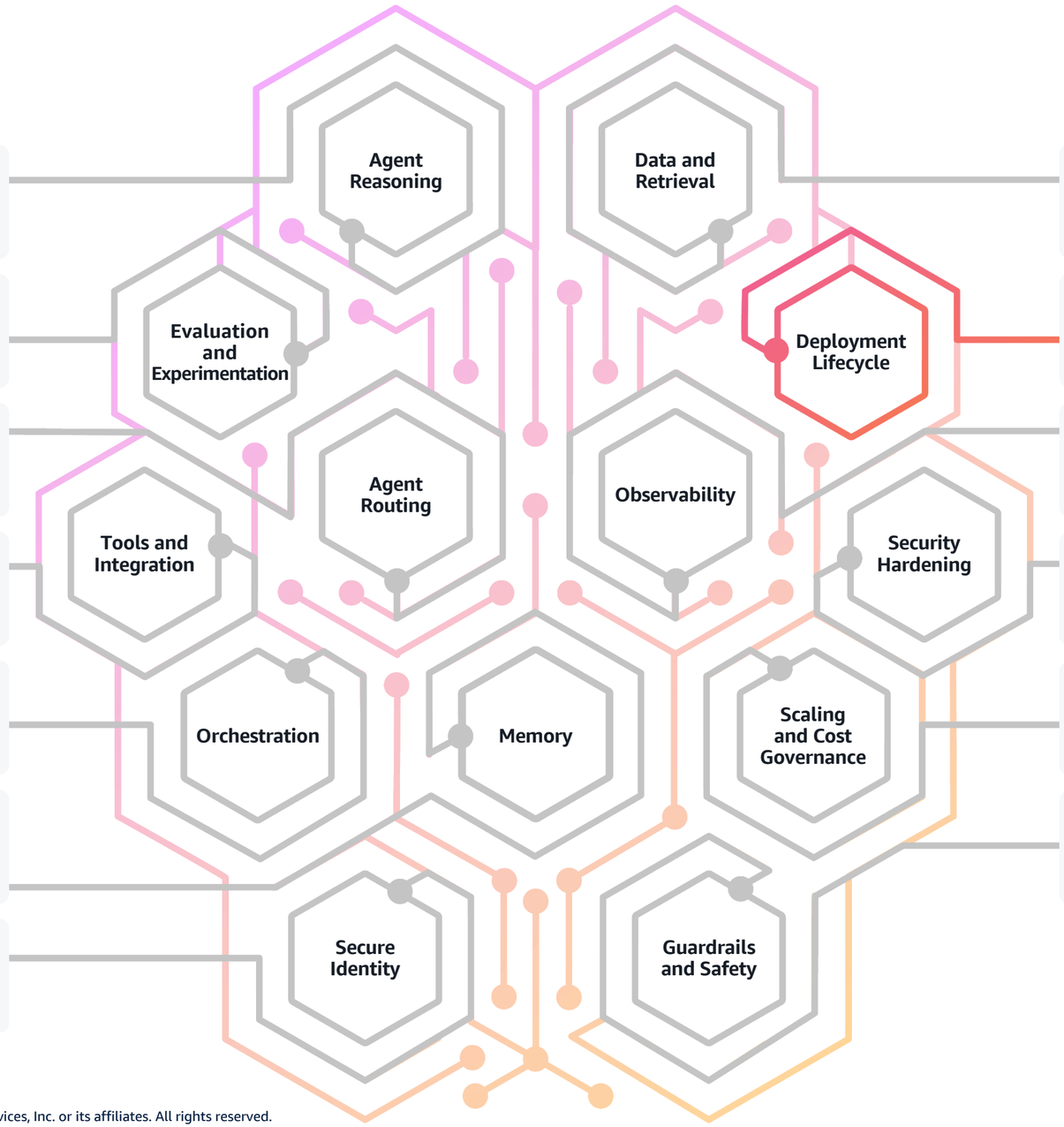
Agent Routing
Classifying inputs and dispatching them to the right agent, model, or workflow path based on intent, cost, and capability matching.

Tools and Integration
Connecting agents to external systems, APIs, live data, and specialized capabilities through tools, MCP servers, and agent-to-agent protocols.

Orchestration
Coordinating networks of specialized agents through a control plane that sequences work, manages shared context, supports human-in-the-loop approval, and handles failures gracefully.

Memory
Designing short-term, long-term, and episodic memory so agents maintain continuity, share context, and learn across sessions and agent boundaries.

Secure Identity
Engineering agent identity distinct from user identity, with least-privilege access, credential vaulting, and full audit trails across every system an agent touches.



Data and Retrieval
Building ingestion, transformation, and lineage pipelines that feed agents accurate enterprise knowledge through dense, sparse, hybrid, and graph-based RAG patterns.

Deployment Lifecycle
Managing agent versioning, prompt promotion, rollback, and evaluation-gated CI/CD across development, staging, and production environments.

Observability
Instrumenting agents with distributed tracing, cost tracking, latency profiling, and anomaly detection to make non-deterministic systems understandable and debuggable.

Security Hardening
Securing compute, networking, containers, and dependencies against prompt injection, secrets exposure, and AI-specific attack vectors.

Scaling and Cost Governance
Optimizing inference capacity, latency, and GPU utilization while managing token budgets, model tiering, caching, and spend attribution at scale.

Guardrails and Safety
Enforcing input/output validation, content filtering, hallucination detection, and policy compliance to keep agent behavior safe and auditable.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Agentic AI tools and capabilities in AWS Marketplace

AGENT REASONING

Claude · cohere · Hugging Face · LangChain · Dify · Progress Agentic RAG · composable

Amazon Bedrock

STRANDS AGENTS SDK

EVALUATION AND EXPERIMENTATION

LangChain · Langfuse · vellum · Weights & Biases · comet LLM · Patronus AI · deepchecks.

Amazon Bedrock AgentCore (Evaluations)

Amazon SageMaker AI

Amazon CloudWatch

AGENT ROUTING

LangChain

AWS Step Functions

Amazon EventBridge

Amazon Bedrock

TOOLS AND INTEGRATION

algolia · mapbox · Deepgram · Kong Konnect · Tyk · TAILWINDS · portkey · SMARTBEAR. · krakenD

AWS Lambda

Amazon API Gateway

Amazon Bedrock AgentCore (Gateway)

ORCHESTRATION

Temporal · orkes · PREFECT · ASTRONOMER

AWS Step Functions

Amazon Bedrock

Amazon MWAA

MEMORY

zilliz · redis · elastic · TiDB Cloud · MongoDB. Atlas · Weaviate · yugabyteDB · Pinecone

Amazon OpenSearch

Amazon MemoryDB

Amazon Bedrock AgentCore (Memory)

SECURE IDENTITY

Ping Identity · okta · CYBERARK

Amazon Cognito

AWS Secrets Manager

Amazon Bedrock AgentCore (Identity)

DATA AND RETRIEVAL

CONFLUENT · databricks · Fivetran · Collibra · Alation · Bigeye · MONTE CARLO · boomi

Amazon Bedrock Knowledge Bases

Amazon S3

AWS Glue

DEPLOYMENT LIFECYCLE

Pulumi · GitLab · Vercel · LaunchDarkly · HashiCorp Vault · Open for Innovation KNIME

AWS CodePipeline

Amazon SageMaker AI

AWS CDK

OBSERVABILITY

DATADOG · honeycomb.io · arize · LangWatch · True Watch · INSTANA an IBM Company · logz.io

Amazon CloudWatch

AWS X-Ray

AWS Distro for OpenTelemetry

SECURITY HARDENING

snyk · Protect AI · Prompt: from SentinelOne · tailscale

Amazon GuardDuty

AWS Security Hub

Amazon Inspector

SCALING AND COST GOVERNANCE

ClickHouse · NVIDIA AI ENTERPRISE · Modal · cast.ai · baseten · anyscale · hydrolix · StormForge

Amazon EKS

AWS Fargate

Amazon Bedrock AgentCore (Runtime)

GUARDRAILS AND SAFETY

Claude · Guardrails AI · Arthur

Amazon Bedrock Guardrails

Amazon Bedrock AgentCore (Policy)

2,800+ Agentic AI tools and capabilities

To help you build, deploy, and operate agentic systems on AWS—available through your AWS account.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Why AWS Marketplace?

Streamline product discovery, procurement, and deployment.

70%

Time savings
for product discovery

60%

Time savings
due to streamlined procurement

30%

Faster time-to-market
due to deployment efficiencies

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Source: “The Total Economic Impact™ Of AWS Marketplace” —a commissioned study by Forrester Consulting on behalf of AWS, May 2025. Results are based on a composite organization comprised of interviewees with experience using AWS Marketplace.

AWS Marketplace offers pricing flexibility

Flexible consumption models

Ease into commitments

Free trials

Get started with a free trial before committing to purchase.

Bring your own license

Migrate to AWS with your existing product licenses. Intended for pre-existing customers.

Usage-based subscriptions

Pay for software and compute capacity by the second or hour with no long-term commitments.

Contracts

Make a monthly, annual, or multi-year payment and receive a discount.

Private offers

Negotiate a custom price with a seller or Channel Partner. Accept the offer in AWS Marketplace.

Flexible payment scheduler

Set a custom billing frequency with your seller.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Key takeaways

The artifact you version is not a trained model. It is the harness.

1 Declare and describe

- Infrastructure code
- A manifest of references
- Let AgentCore version what it can

2 Gate honestly

- Hard vs statistical
- Relative to a named baseline
- Calibrate, then version the ruler

3 Ship for what will not revert

- Shadow, then slow canary
- Flags for capabilities
- Stamp side effects, namespace memory

Next: Module 12 covers security and observability. The attack surface a harness exposes, prompt injection, audit for agent actions.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Next steps

MLOps and the agent development lifecycle

Read the companion guide: Release the harness as one versioned object: infrastructure as code, a content-addressed manifest, evaluation gates that survive non-determinism, progressive delivery, and rollback that accounts for what an agent already did.

Building agentic systems on AWS

Explore the 14-module workshop series. Learn to design, orchestrate, and scale multi-agent systems – from foundational patterns to production deployment – with structured tutorials and vetted AWS Marketplace integrations.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank you!

aws marketplace

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.